

Nama : Asmaratunga Avenda Darma Putra
Nim : 24102006
Matkul : Pemrograman Web
Tugas : Praktikum 10 (Autentifikasi)

ANALISIS DAN IMPLEMENTASI KEAMANAN PADA APLIKASI TODO LIST BERBASIS LARAVEL

Pendahuluan

Keamanan merupakan salah satu aspek penting dalam pengembangan aplikasi web. Sebuah aplikasi yang tidak menerapkan mekanisme keamanan yang baik berpotensi mengalami berbagai serangan seperti Cross Site Scripting (XSS), SQL Injection, pencurian sesi pengguna, hingga akses tidak sah terhadap data. Oleh karena itu, pada pengembangan aplikasi Todo List berbasis Laravel ini telah diterapkan berbagai mekanisme keamanan yang mengacu pada praktik pengembangan web modern.

Dokumen ini menjelaskan implementasi keamanan yang telah diterapkan pada aplikasi Todo List beserta mitigasi terhadap potensi kerentanan yang umum ditemukan pada aplikasi web.

1. Validasi Input Pengguna

Tujuan

Validasi input bertujuan untuk memastikan bahwa seluruh data yang diterima aplikasi sesuai dengan format yang telah ditentukan sebelum diproses atau disimpan ke dalam database.

Tanpa validasi yang baik, pengguna dapat mengirimkan data yang tidak sesuai, data kosong, maupun data berbahaya yang dapat menyebabkan kerusakan sistem atau celah keamanan.

Implementasi

Aplikasi menerapkan dua lapisan validasi, yaitu:

a. Client-Side Validation

Validasi dilakukan langsung pada browser menggunakan atribut HTML5 seperti:

- required
- minlength
- maxlength
- type="email"

Nama : Asmaratunga Avenda Darma Putra
Nim : 24102006
Matkul : Pemrograman Web
Tugas : Praktikum 10 (Autentifikasi)
Contoh:

```
<input type="text" name="task" required maxlength="255">
```

Dengan mekanisme ini pengguna akan memperoleh umpan balik secara langsung apabila terdapat data yang belum sesuai sebelum formulir dikirim ke server.

b. Server-Side Validation

Validasi utama dilakukan pada sisi server menggunakan fitur validasi bawaan Laravel.

Contoh:

```
$request->validate([  
    'task' => 'required|max:255',  
    'is_done' => 'boolean'  
]);
```

Validasi ini diterapkan pada:

- Method store()
- Method update()
- Method login()

Keuntungan dari validasi server-side adalah seluruh data tetap diperiksa meskipun pengguna mencoba melewati validasi browser menggunakan tools tertentu.

2. Perlindungan Terhadap Cross Site Scripting (XSS)

Potensi Risiko

Cross Site Scripting (XSS) merupakan serangan yang terjadi ketika penyerang memasukkan script berbahaya ke dalam aplikasi dan script tersebut dijalankan pada browser pengguna lain.

Sebagai contoh, seorang penyerang dapat menyimpan kode berikut pada kolom tugas:

```
<script>alert('Hacked')</script>
```

Jika aplikasi menampilkan data tersebut tanpa sanitasi, maka script akan dieksekusi oleh browser pengguna lain.

Nama : Asmaratunga Avenda Darma Putra
Nim : 24102006
Matkul : Pemrograman Web
Tugas : Praktikum 10 (Autentifikasi)

Implementasi pada Aplikasi

Aplikasi menggunakan Blade Templating Engine yang merupakan sistem template bawaan Laravel.

Contoh penampilan data:

```
{{ $d->task }}
```

Secara otomatis Blade akan melakukan proses escaping menggunakan fungsi `htmlspecialchars()` sehingga karakter khusus akan dikonversi menjadi bentuk aman.

Contoh:

Input pengguna:

```
<script>alert('Hacked')</script>
```

Output yang ditampilkan:

```
&lt;script&gt;alert('Hacked')&lt;/script&gt;
```

Akibatnya script tidak dijalankan oleh browser dan hanya tampil sebagai teks biasa.

Dampak Keamanan

Penerapan mekanisme ini mampu mengurangi risiko:

- Stored XSS
- Reflected XSS
- Session hijacking melalui script
- Pencurian cookie pengguna

3. Perlindungan Terhadap SQL Injection

Potensi Risiko

SQL Injection merupakan salah satu serangan paling berbahaya pada aplikasi web. Serangan ini terjadi ketika input pengguna dimasukkan secara langsung ke dalam query SQL tanpa proses sanitasi.

Contoh query yang rentan:

```
SELECT * FROM todos WHERE task = '$search'
```

Nama : Asmaratunga Avenda Darma Putra

Nim : 24102006

Matkul : Pemrograman Web

Tugas : Praktikum 10 (Autentifikasi)

Penyerang dapat memasukkan karakter tertentu sehingga struktur query berubah dan memungkinkan akses data yang tidak sah.

Implementasi pada Aplikasi

Aplikasi menggunakan Laravel Eloquent ORM dan Query Builder.

Contoh kode pencarian:

```
$query->where('task', 'LIKE', '%' . $search . '%');
```

Laravel secara otomatis menggunakan prepared statement dan parameter binding.

Di belakang layar Laravel menghasilkan query yang aman seperti:

```
SELECT * FROM todos WHERE task LIKE ?
```

Nilai parameter kemudian dikirim secara terpisah sehingga tidak dapat mengubah struktur query.

Dampak Keamanan

Mekanisme ini melindungi aplikasi dari:

- SQL Injection
- Manipulasi query database
- Penghapusan data secara ilegal
- Akses tidak sah terhadap tabel database

4. Keamanan Password dan Autentikasi Pengguna

Tujuan

Password merupakan data sensitif yang tidak boleh disimpan dalam bentuk teks biasa (plain text).

Jika database bocor, seluruh akun pengguna dapat langsung diambil alih oleh pihak yang tidak bertanggung jawab.

Implementasi

Laravel secara otomatis menggunakan algoritma hashing Bcrypt.

Contoh:

Nama : Asmaratunga Avenda Darma Putra
Nim : 24102006
Matkul : Pemrogaman Web
Tugas : Praktikum 10 (Autentifikasi)
Hash: :make(\$password);

Hasil penyimpanan password:

\$2y\$12\$3LhBf...

Password yang telah di-hash tidak dapat dikembalikan ke bentuk aslinya karena menggunakan metode one-way hashing.

Pada saat login, Laravel menggunakan:

Auth::attempt()

untuk membandingkan password yang dimasukkan dengan hash yang tersimpan pada database.

Dampak Keamanan

Penerapan hashing memberikan perlindungan terhadap:

- Kebocoran password pengguna
 - Pengambilalihan akun akibat pencurian database
 - Penyimpanan password dalam bentuk teks biasa
-

5. Manajemen Session dan Perlindungan CSRF

Potensi Risiko

Cross Site Request Forgery (CSRF) merupakan serangan yang memanfaatkan sesi login pengguna untuk mengirim permintaan tanpa sepengetahuan pengguna.

Contohnya, pengguna yang masih login dapat dipaksa mengirim permintaan penghapusan data melalui situs berbahaya.

Implementasi

Laravel menyediakan token CSRF secara otomatis.

Setiap form yang menggunakan metode:

- POST
- PUT
- PATCH
- DELETE

Nama : Asmaratunga Avenda Darma Putra
Nim : 24102006
Matkul : Pemrograman Web
Tugas : Praktikum 10 (Autentifikasi)
menyertakan:

@csrf

Contoh:

```
<form method="POST">  
    @csrf  
</form>
```

Saat formulir dikirim, Laravel akan memverifikasi token tersebut.

Jika token tidak valid, permintaan akan ditolak.

Dampak Keamanan

Fitur ini melindungi aplikasi dari:

- Pemalsuan permintaan
 - Penghapusan data tanpa izin
 - Manipulasi data oleh pihak ketiga
-

6. Pengendalian Hak Akses (Access Control)

Tujuan

Fitur Todo List hanya boleh digunakan oleh pengguna yang telah berhasil melakukan autentikasi.

Implementasi

Aplikasi menggunakan middleware autentikasi Laravel.

Contoh:

```
Route::middleware('auth')->group(function () {  
    // route todo  
});
```

Atau:

```
->middleware('auth')
```

Ketika pengguna yang belum login mencoba mengakses halaman Todo List, Laravel secara otomatis mengarahkan pengguna ke halaman login.

Nama : Asmaratunga Avenda Darma Putra
Nim : 24102006
Matkul : Pemrograman Web
Tugas : Praktikum 10 (Autentifikasi)

Dampak Keamanan

Mekanisme ini mencegah:

- Akses tidak sah
 - Manipulasi data oleh pengguna anonim
 - Penggunaan fitur tanpa autentikasi
-

7. Penanganan Error yang Aman

Potensi Risiko

Pesan kesalahan yang terlalu detail dapat membocorkan informasi internal sistem seperti:

- Struktur folder server
- Query database
- Nama tabel
- Konfigurasi aplikasi

Informasi tersebut dapat dimanfaatkan penyerang untuk melakukan eksploitasi lanjutan.

Implementasi

Pada lingkungan produksi digunakan konfigurasi:

```
APP_DEBUG=false
```

Dengan konfigurasi ini Laravel tidak menampilkan stack trace maupun informasi teknis kepada pengguna.

Selain itu, pesan validasi dibuat lebih ramah pengguna sehingga hanya menampilkan informasi yang diperlukan.

Contoh:

Kolom tugas wajib diisi.

bukan:

```
SQLSTATE[23000] Integrity constraint violation...
```

Dampak Keamanan

Implementasi ini mengurangi risiko:

Nama : Asmaratunga Avenda Darma Putra
Nim : 24102006
Matkul : Pemrograman Web
Tugas : Praktikum 10 (Autentifikasi)

- Information Disclosure
- Reconnaissance Attack
- Eksploitasi berbasis error message

8. Keamanan Endpoint API

Implementasi

Aplikasi menyediakan endpoint API:

/api/todos

Data dikirim menggunakan format JSON yang terstruktur.

Contoh:

```
{  
  "id": 1,  
  "task": "Belajar Laravel",  
  "is_done": false  
}
```

Format ini lebih aman dan mudah diintegrasikan dengan aplikasi lain seperti:

- Mobile Application
- Single Page Application (SPA)
- Sistem pihak ketiga

Pengembangan Selanjutnya

Jika API digunakan secara publik atau terpisah dari aplikasi utama, disarankan menambahkan:

- Laravel Sanctum
- Laravel Passport
- Token Authentication
- Rate Limiting

untuk meningkatkan keamanan akses API.

Nama : Asmaratunga Avenda Darma Putra
Nim : 24102006
Matkul : Pemrograman Web
Tugas : Praktikum 10 (Autentifikasi)

Kesimpulan

Berdasarkan hasil implementasi yang telah dilakukan, aplikasi Todo List telah menerapkan berbagai mekanisme keamanan dasar yang direkomendasikan dalam pengembangan aplikasi web modern menggunakan Laravel. Penerapan validasi input, perlindungan XSS, pencegahan SQL Injection, hashing password, CSRF protection, middleware autentikasi, penanganan error yang aman, serta pengelolaan API yang terstruktur menunjukkan bahwa aplikasi telah dirancang dengan memperhatikan aspek keamanan sejak tahap pengembangan.

Meskipun demikian, keamanan aplikasi merupakan proses yang berkelanjutan. Untuk pengembangan berikutnya, disarankan menambahkan fitur keamanan lanjutan seperti Rate Limiting, Two-Factor Authentication (2FA), Audit Log Aktivitas Pengguna, serta autentikasi berbasis token menggunakan Laravel Sanctum atau Passport agar aplikasi semakin siap digunakan pada lingkungan produksi yang lebih kompleks.